Tecnologías IoT para la Industria 4.0 · Bloque 0

Práctica D4 · Conteneriza tu microservicio

Microservicio Spring Boot y su base de datos PostgreSQL desplegados con Docker Compose

Alumno	Manu Martín
Curso	2.º DAM
Fecha	31 de octubre de 2026
Objetivo	Desplegar con Docker Compose un microservicio Spring Boot con su base de datos
Criterios	CE6.1, CE6.2
Proyecto	`04-springboot` (microservicio `inventario`, que el enunciado permite usar)
Entrega	`Dockerfile`, `compose.yaml`, `.env.example` y capturas
Entorno	Windows 11, Docker Desktop (motor 29.8.2, Compose 5.6.0), PowerShell

Índice

Parte 1.1 · Crear el .env y arrancar 3
Parte 1.2 · Probar la API 3
Parte 1.3 · Persistencia: down/up y down -v 4
Parte 1.4 · Entrar en la base de datos sin publicar su puerto 4
Parte 2 · Aplicado a un proyecto propio 5
Comprueba que funciona 6
Reto · Adminer 6
Estado final 7

Parte 1.1 · Crear el .env y arrancar

```
Copy-Item .env.example .env
(Get-Content .env) -replace "DB_CLAVE=.*", ("DB_CLAVE=" + [guid]::NewGuid().ToString("N")) | Set-
Content .env
docker compose up -d --build --wait
```

La clave de la base de datos se genera aleatoriamente y no se muestra en ninguna captura. La primera construcción tardó unos 4 minutos: descarga las imágenes de Maven y del JRE, y las dependencias (`dependency:go-offline`, 71 s). La compilación del código solo tardó 8 s. Con `--wait`, el comando no termina hasta que `db` y `api` están `Healthy`, y la API arranca después de la base de datos.

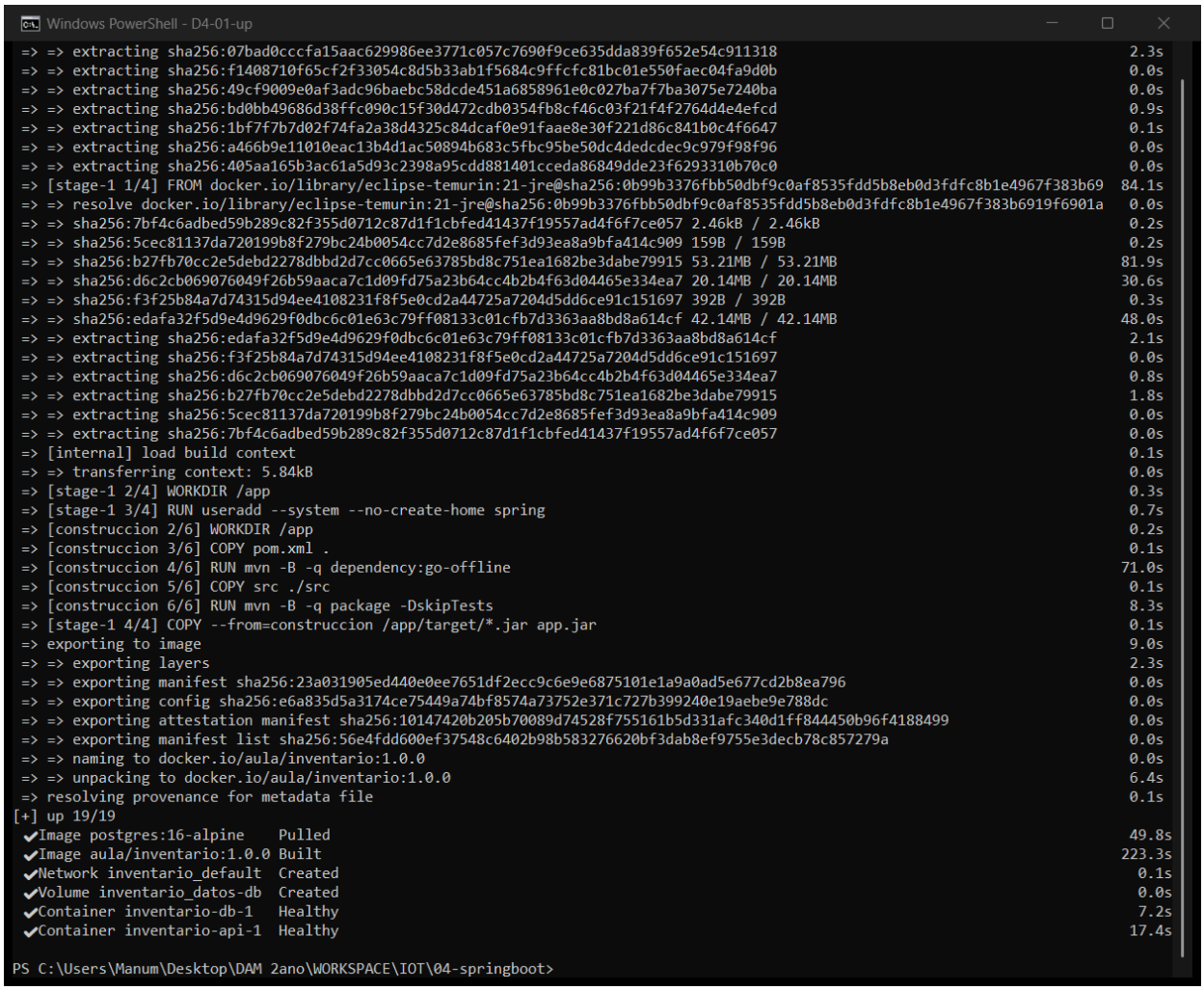


Figura 1. *Construcción multietapa y arranque ordenado: db y api quedan Healthy.*

Parte 1.2 · Probar la API

```
curl.exe -s http://localhost:8080/api/dispositivos
curl.exe -s http://localhost:8080/actuator/health
Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/dispositivos -ContentType
"application/json" -Body '{"nombre":"deposito-3","ubicacion":"patio","tipo":"nivel"}'
```

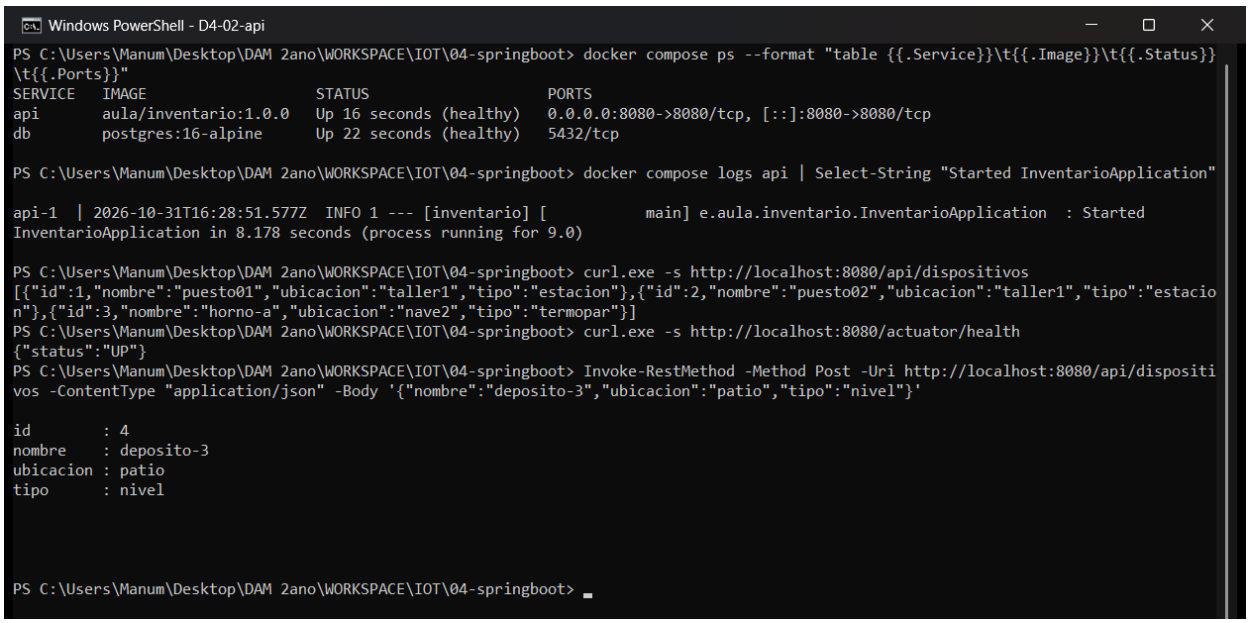


Figura 2. *La API devuelve los datos iniciales, Actuator responde UP y se crea un dispositivo nuevo (id 4).*

Parte 1.3 · Persistencia: down/up y down -v

Con `docker compose down` se borran los contenedores y la red, pero no el volumen `datos-db`. Al volver a arrancar, el dispositivo 4 sigue ahí. Con `docker compose down -v` también se borra el volumen: la base de datos se crea vacía y la aplicación vuelve a cargar solo los tres dispositivos iniciales.

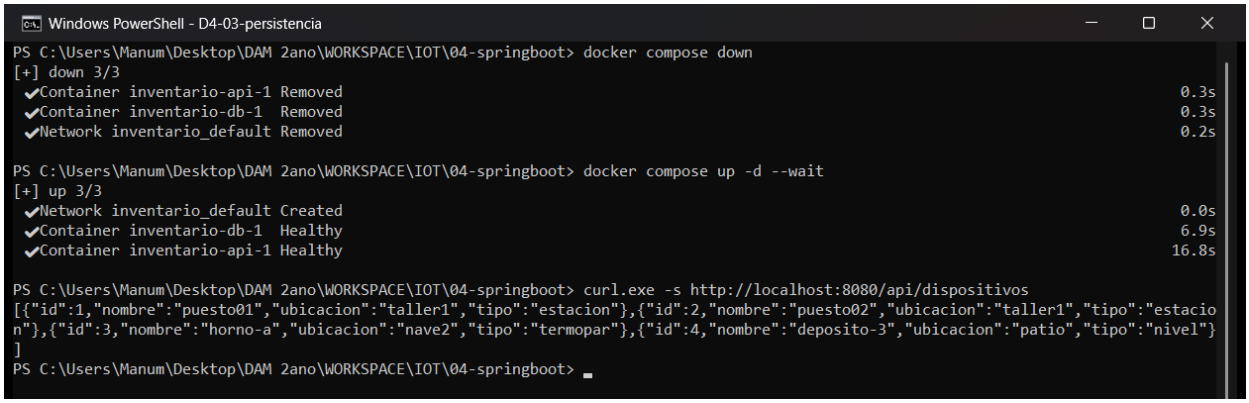


Figura 3. *Tras down y up, el dispositivo creado sigue en la base de datos.*

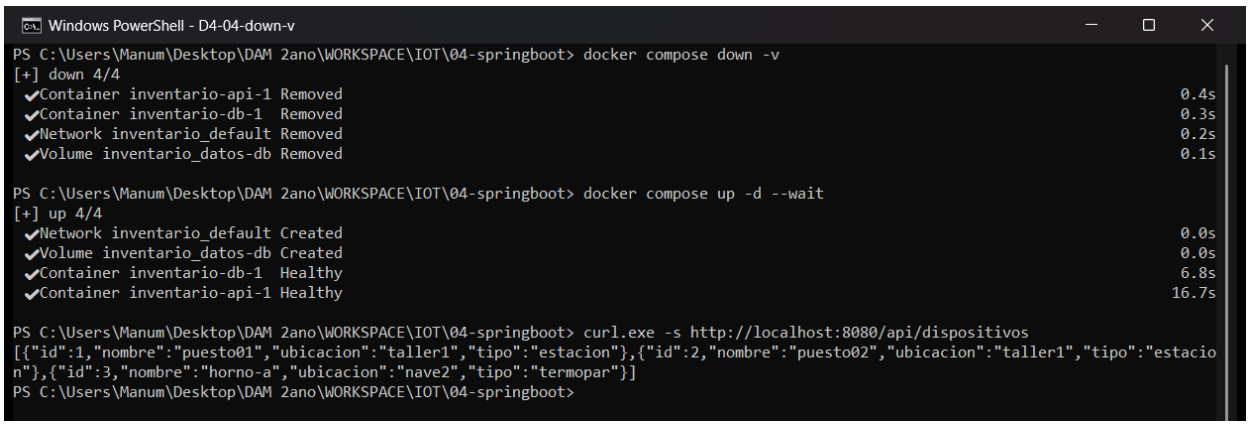


Figura 4. *down -v borra el volumen: solo quedan los datos iniciales.*

Parte 1.4 · Entrar en la base de datos sin publicar su puerto

```
docker compose exec db psql -U inventario -c "select * from dispositivo"
docker compose port db 5432
```

`exec` ejecuta `psql` dentro del contenedor, así que no hace falta publicar el puerto 5432. `docker compose port db 5432` devuelve `:0`: el puerto no está publicado en el equipo.

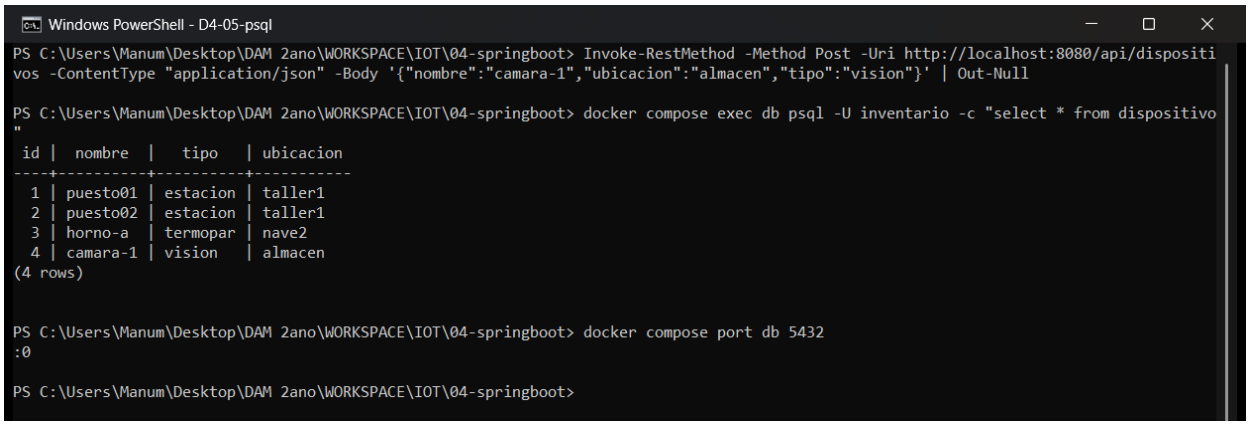


Figura 5. *Consulta con psql dentro del contenedor; el puerto 5432 no está publicado.*

Parte 2 · Aplicado a un proyecto propio

Los pasos de la parte 2 se han aplicado y comprobado sobre `04-springboot`:

- **Dockerfile y .dockerignore:** el Dockerfile multietapa compila con `maven:3.9-eclipse-temurin-21` y ejecuta con `eclipse-temurin:21-jre`, coherente con `java.version` 21 del `pom.xml`. El `.dockerignore` excluye `target/`, `.idea/` y `.env`.
- **Actuator:** está en el `pom.xml` y `application.properties` publica solo `health` (`management.endpoints.web.exposure.include=health`).
- **compose.yaml:** base de datos PostgreSQL con volumen `datos-db`, healthchecks en los dos servicios (`pg_isready` y `/actuator/health`), variables en `.env` y `depends_on` con `service_healthy`.
- **Arranca desde cero con un solo comando:** sí, la Figura 4 parte de un `down -v`.
- **.gitignore:** el proyecto no tenía, así que se ha creado uno que excluye `.env` y `target/`.

La imagen final ocupa 561 MB; la etapa de construcción, 1,16 GB. Solo se despliega la primera. En el repositorio no hay contraseñas reales: `.env.example` lleva un valor de ejemplo, el compose usa `${DB_CLAVE}` y `application.properties` solo tiene el valor `cambiame` para trabajar desde el IDE, que en Docker se sustituye por la variable `SPRING_DATASOURCE_PASSWORD`.

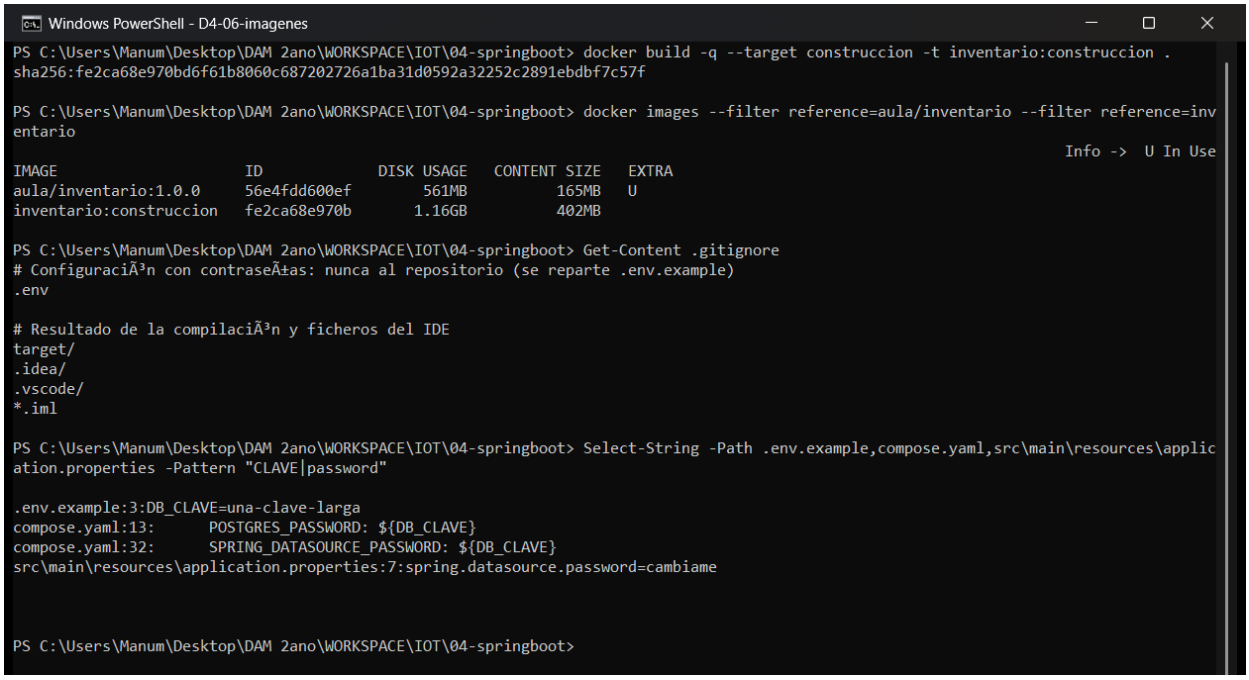


Figura 6. *Imagen final frente a la etapa de construcción, .gitignore y búsqueda de claves en los ficheros del proyecto.*

Comprueba que funciona

- **docker compose ps muestra los dos servicios healthy:** sí (Figuras 1 y 2).
- **La base de datos no aparece con puertos publicados:** sí: solo `5432/tcp` interno, sin `0.0.0.0:` (Figuras 2 y 5).
- **Sin contraseñas en el repositorio:** sí: están en `.env`, y `.env` está en `.gitignore`.
- **Se puede arrancar sin Java instalado:** la compilación ocurre en la etapa `construccion` de Docker, así que basta con Docker.

Reto · Adminer

Se añadió `adminer:4.8.1` como servicio, publicado en el puerto 8081 y con `ADMINER_DEFAULT_SERVER: db`, para administrar PostgreSQL desde el navegador. Es otra forma de llegar a la base de datos sin publicar su puerto: Adminer está en la misma red y entra por el nombre `db`. Los otros dos retos (un segundo microservicio y publicar la imagen en Docker Hub) no se han hecho: el segundo requiere escribir otro servicio y el tercero, una cuenta personal de Docker Hub.

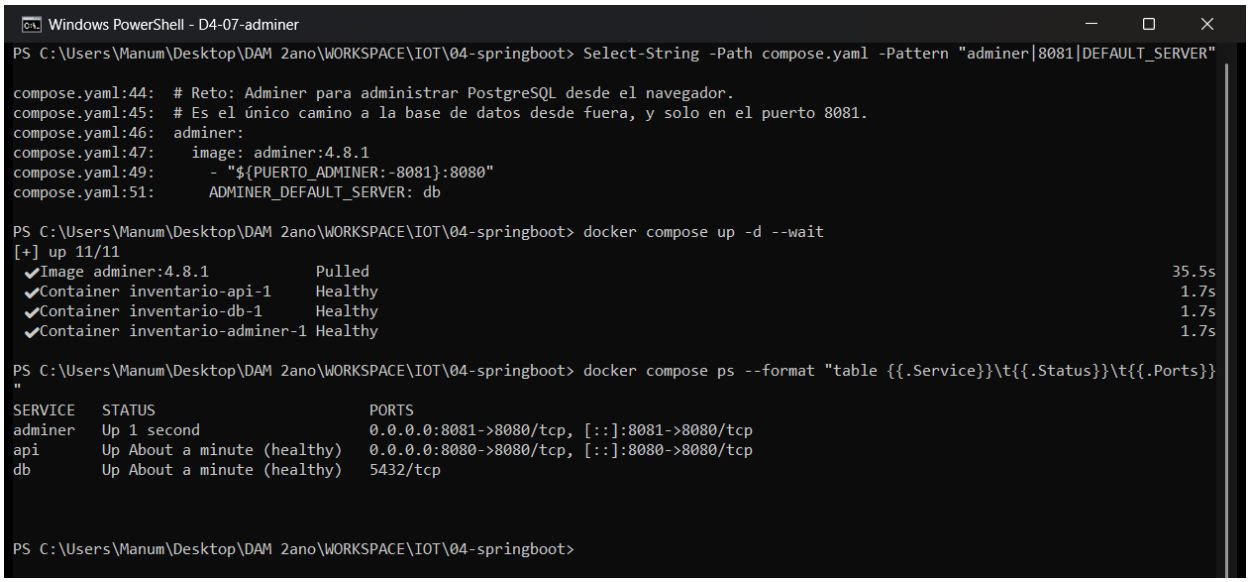


Figura 7. *Adminer como tercer servicio, en el puerto 8081.*



Figura 8. *Pantalla de acceso de Adminer, con el servidor db ya rellenado.*

Estado final

`04-springboot` tiene un `.gitignore` nuevo y Adminer en el compose (puerto 8081). Al terminar no queda ningún contenedor en marcha; los datos siguen en el volumen `inventario_datos-db`.